

Software Prototyping in Software Engineering

Course: Software Engineering

Instructor: Adel Vahdati

Learning Objectives:

- Understand what software prototyping is and why it's used.
 - Explore types of prototypes and their purposes.
 - Analyze benefits, risks, and best practices.
 - Examine real-world applications and case studies.
-

Slide 2: What is Software Prototyping?

Definition:

A software prototype is an early approximation of a software system built to explore ideas, validate assumptions, and gather user feedback before full-scale development.

Key Characteristics:

- Incomplete or simplified functionality.
- Focus on learning and validation, not delivery.
- Iterative and user-centered.

Reference: Sommerville, I. (2020). *Software Engineering* (10th ed.). Pearson.

Slide 3: Why Prototype?

Motivations:

- Clarify and validate requirements early.
- Improve stakeholder communication.
- Reduce technical and design risks.
- Enhance user interface (UI) and user experience (UX).
- Save time and cost through early feedback.

Example: A banking app prototype allows users to test navigation and security flows before full coding begins.

Reference: Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*.

Slide 4: The Prototyping Process

Phases:

1. **Requirements Identification**
 - Define the purpose and scope of the prototype.
2. **Quick Design**
 - Sketch interfaces and system flow.
3. **Build Prototype**
 - Develop mock-ups or partial systems.
4. **User Evaluation**
 - Collect feedback and adjust requirements.
5. **Refinement**
 - Iterate until clarity is achieved.
6. **Final Product Development**
 - Either discard or evolve prototype into final system.

Diagram: Prototyping feedback loop (requirements → design → prototype → evaluation → refinement)

Slide 5: Types of Prototypes

Type	Description	Use Case Example
Throwaway	Quickly built and discarded after validation.	Early UI sketches for requirement gathering.
Evolutionary	Continuously refined into the final system.	Agile web app development.
Incremental	Prototypes for separate modules later integrated.	Modular IoT systems.
Extreme/Rapid	Uses tools/frameworks for fast iterations.	Mobile app mock-ups in Figma or Sketch.
Horizontal/Vertical	Breadth vs. depth of prototype features.	Horizontal: menu-only mockup; Vertical: deep login flow.

References: GeeksforGeeks (2023), IEEE Software Standards.

Slide 6: Advantages and Disadvantages

Advantages:

- Early detection of requirement misunderstandings.
- Encourages user involvement.
- Reduces rework during later phases.
- Aids in training and documentation.

Disadvantages:

- Can lead to poorly structured code if evolved directly.
 - Users may mistake prototypes for finished products.
 - May ignore non-functional requirements (e.g., performance).
 - Risk of endless iteration cycles.
-

Slide 7: Best Practices

Guidelines for Effective Prototyping:

- Define **clear goals**: What do you want to test or learn?
- Keep it **simple and focused**: avoid unnecessary features.
- Apply **time-boxing** to limit iterations.
- Use appropriate **fidelity**: low-fi for ideas, high-fi for usability testing.
- Separate prototype from final codebase.
- Document decisions and feedback.

Tools: Figma, Balsamiq, Axure, Adobe XD, InVision.

Slide 8: When to Use Prototyping

Prototyping is suitable when:

- Requirements are uncertain or evolving.
- High user interaction is expected.
- System is novel or high-risk.
- Used within the **Spiral** or **Iterative** models.

Not ideal when:

- Requirements are stable and well-understood.
 - Time and cost constraints are rigid.
-

Slide 9: Case Study 1 — NASA Mission Control Software

Context: NASA used prototyping for its ground control systems (1980s–1990s).

Approach: Evolutionary prototyping with iterative feedback from engineers.

Outcome: Reduced communication errors, improved system usability, and avoided critical requirement gaps.

Reference: Boehm, B. (1988). *A Spiral Model of Software Development and Enhancement*, IEEE Computer.

Slide 10: Case Study 2 — Google’s Gmail Interface Prototype

Context: Gmail’s early design underwent multiple UI/UX prototypes before launch.

Approach: Rapid and horizontal prototyping to test navigation and performance.

Outcome: Improved usability and reduced user onboarding time.

Lesson: User-centered design through prototyping leads to higher adoption rates.

Slide 11: Summary & Discussion

Key Takeaways:

- Prototyping enhances requirement clarity, design quality, and stakeholder satisfaction.
- Choose the type of prototype based on project goals.
- Avoid the trap of evolving poor-quality prototypes.

Discussion Questions:

1. When should a prototype be discarded vs. evolved?
 2. How can prototyping fit into Agile or DevOps workflows?
 3. What tools would you choose for your capstone project and why?
-

Slide 12: References

1. Sommerville, I. (2020). *Software Engineering* (10th ed.). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
3. Boehm, B. (1988). *A Spiral Model of Software Development and Enhancement*, IEEE Computer.
4. IEEE Std 1058-1998: *Software Project Management Plans*.
5. GeeksforGeeks (2023). *Software Prototyping Model*.
6. Studocu. *Software Engineering Notes — Prototyping Model*.